Introduction to High-Performance and Parallel... Computing

Today's goals

Module 1
High-Performance Computing (HPC) for Non-Computer Scientists

Module 2
Nuts and Bolts of HPC

Module 3
Basic Parallelism

Simple Application Timing
Simple Application Timing
Video • 3 min

Serial vs. Parallel Processing
Serial vs. Parallel Processing - Part 1
Video • 3 min
Serial vs. Parallel Processing - Part 2
Video • 5 min

Parallel Memory Models
Parallel Memory Models
Video • 5 min

Data vs. Task Parallelism
Data vs. Task Parallelism
Video • 5 min

High Throughput Computing
High Throughput Computing
Video • 4 min

Module Assignments
Week 3 Quiz
Graded Assignment • Grade: 100%
Simple Application Timing
Programming Assignment • 3h

Module 4
Evaluating Parallel Program Performance

How to Parallelize Code
How to Parallelize Code
Video • 6 min

Speedup and Parallel Efficiency
Speedup and Parallel Efficiency
Video • 4 min

Scalability
Scalability
Video • 4 min

Limits to Parallel Performance
Limits to Scaling
Video • 3 min

Module Assignments
Week 4 Quiz
Graded Assignment • 30 min
Strong Scaling Study
Programming Assignment • 1h
Weak Scaling Study
Programming Assignment • 1h

Course Summary
Summary of This Course
Reading • 10 min

Programming assignment

Simple Application Timing

Start your mini cluster. Work this week in "projects/week3-timing"

What to expect
Due Sep 18, 11:59 PM IST
Unlimited attempts

Instructions | My submissions | Discussions

Let's practice timing a loop within a short program. To begin the exercise, copy one of the timing examples in the **projects/week3-timing** directory in your JupyterLab environment (*timing.{f90,cpp,py}) and rename it as **timing_sol.{f90,cpp,py}**. Then, modify your solution file as follows:

1. Update the calculation for r to compute r = a * b + c where r, a, b, and c are all one dimensional, double-precision arrays of length nx.
2. Initialize the values a, b, c to a = 1.0, b = 2.0, and c = 5.0
3. Modify the script to accept an additional command-line argument, "**-nx**", so the program may be called as **./program_name -nt M -nx N**, where M is the number of loops to perform and N is the size of the array.
4. Your script should report the values:
   a. Number of points
   b. Number of iterations
   c. Elapsed time
   d. Elapsed time per iteration
   e. Elapsed time per iteration per point

Test your script using the appropriate commands below:

Fortran90

- Compile: gfortran -Ofast -Wall timing_sol.f90 -o timing_sol.fexe
- Execute: ./timing_sol.fexe -nt N -nx M

C++

- Compile: g++ -Ofast -Wall timing_sol.cpp -o timing_sol.cexe
- Execute: ./timing_sol.cexe -nt N -nx M

Python

- Execute: python timing_sol.py -nt N -nx M

Once you are satisfied with your script, compress your solution file using the command **tar -cvf timing_sol.tar timing_sol.{f90,cpp,py}**

Submit your compressed file for autograding. The autograder will compile and run your script and compare the "Elapsed time per iteration per point" value to the solution. Note, you need to include all 5 reported values in your solution.

Discussion

Once you have received feedback for your script, run the program with -nx set to the following values:

- 1000
- 7000
- 100000

Choose -nt so that the program runs at least for 1 second.

Examine how the elapsed time changes as you keep -nt or -nx constant. We will explain more about why this is in our course about efficient programming. There is a discussion question about the results.

Report an issue

Go to next item →